

Blatt 11

Software Qualität

Qualitätsmetriken, GQM und objektorientierte Disharmonien

Luis Staudt

Aufgabe 1: Qualitätsmetriken und GQM

Teil 1: Theoretische und methodische Grundlagen

Was ist eine (Qualitäts-)Metrik?

Eine Metrik bildet eine Eigenschaft eines Software-Artefakts oder -Prozesses auf einen Zahlenwert ab und macht sie dadurch mess- und vergleichbar. Eine Qualitätsmetrik misst speziell Eigenschaften, die mit der Software-Qualität (z. B. Wartbarkeit, Zuverlässigkeit) zusammenhängen. Man unterscheidet:

- **Direkte Metriken:** ohne weitere Größen messbar (z. B. Lines of Code).
- **Indirekte/abgeleitete Metriken:** aus mehreren Größen berechnet (z. B. Fehlerdichte = Fehler / KLOC).

Jede Metrik besitzt eine *Skala* (nominal, ordinal, intervall, ratio, absolut), die festlegt, welche Vergleiche und Rechenoperationen zulässig sind.

Kategorien von Metriken

Metriken werden üblicherweise danach unterschieden, was gemessen wird:

Kategorie	Misst	Beispiele
Produktmetriken	Eigenschaften des Artefakts (Code, Entwurf, Docu)	LOC, zyklomatische Komplexität (McCabe), CK-Metriken (WMC, DIT, CBO, LCOM), Testabdeckung, Fehlerdichte
Prozessmetriken	Eigenschaften des Entwicklungs- und Wartungsprozesses	Durchlaufzeit, Fehlerbehebungsrate, Review-Abdeckung, Deployment-Frequenz, MTTR, Change-Failure-Rate

Kategorie	Misst	Beispiele
Projektmetriken	Eigenschaften des Projektmanagements	Aufwand (Personentage), Kosten, Termintreue, Teamgröße, Velocity, Restaufwand

Eigenschaften sinnvoller Metriken

- **Validität:** misst tatsächlich die gemeinte Eigenschaft.
- **Objektivität & Reproduzierbarkeit:** Ergebnis ist unabhängig vom Messenden und wiederholbar.
- **Vergleichbarkeit:** Werte sind über Projekte/Zeit hinweg gegenüberstellbar.
- **Wirtschaftlichkeit:** mit vertretbarem Aufwand (möglichst automatisiert) erhebbar.
- **Verständlichkeit & Zielbezug:** leicht interpretierbar und auf ein konkretes Ziel bezogen.
- **Rechtzeitigkeit & Robustheit:** früh verfügbar und nicht leicht manipulierbar.

Vergleichbarkeit und Bewertung von Metriken

Ein einzelner Messwert ist für sich genommen selten aussagekräftig. Erst durch **Bezugsgrößen** wird er bewertbar:

- **Schwellenwerte (Thresholds):** Soll-/Grenzwerte als Ampel.
- **Baselines/Benchmarks:** Vergleich mit eigenem Ausgangswert oder Branchenwerten.
- **Trends:** Entwicklung über die Zeit (Verbesserung/Verschlechterung).
- **Normalisierung:** z. B. pro KLOC, um Größeneffekte zu entfernen.

Die zulässige Auswertung hängt von der Skala ab (z. B. Mittelwert nur ab Intervallskala). Wichtig ist, stets **Metrik-Sätze** statt Einzelwerte zu betrachten, da einzelne Metriken zu Fehlsteuerung führen können (Goodhart: "a measure that becomes a target ceases to be a good measure").

Aufbau und Funktionsweise des GQM-Ansatzes

Goal–Question–Metric (Basili & Rombach) ist ein **zielorientierter Top-down-Ansatz**, der Messungen aus den Qualitätszielen ableitet – es wird also nur gemessen, was zur Beantwortung einer konkreten Frage beiträgt. Er besteht aus drei Ebenen:

1. **Goal (konzeptionell)**: das Mess-/Qualitätsziel.
2. **Question (operational)**: Fragen, die prüfen, ob das Ziel erreicht wird.
3. **Metric (quantitativ)**: Metriken, die die Fragen beantworten.

Ziele werden mit einer **Schablone** präzisiert:

Analysiere <Objekt> zum Zweck <Absicht> bezüglich <Qualitätsaspekt> aus Sicht <Betrachter> im Kontext <Umgebung>.

Zusammenhang zwischen Zielen, Fragen und Metriken

GQM bildet eine **Baumstruktur**: Ein Ziel wird durch mehrere Fragen verfeinert, jede Frage durch eine oder mehrere Metriken beantwortet. Eine Metrik kann mehreren Fragen dienen.

Ziel → Fragen → Metriken

Die Definition erfolgt **top-down** (vom Ziel zur Metrik), die Interpretation **bottom-up**: gemessene Werte beantworten die Fragen, deren Antworten zeigen, ob das Ziel erreicht ist.

Teil 2: Messkonzept für “Schwarzwald-Entdecker”

Die Anwendung. “Schwarzwald-Entdecker” ist ein Web-Portal (auf gängigen Browsern und mobilen Geräten nutzbar), über das Kunden Ausflüge und Aktivitäten im Schwarzwald planen, auswählen, buchen und bezahlen sowie ihre Buchungen einsehen, ändern und stornieren können. Kernbausteine sind der *Ausflug-Konfigurator* (Filterung passender Angebote nach Präferenzen, Planung kollisionsfreier Mehrtages-/Mehrfachtouren) sowie Registrierung/Login, Buchung mit Verfügbarkeitsprüfung und Bestätigung und eine Buchungshistorie. Laut Anforderung soll das Portal **benutzerfreundlich, zuverlässig, hochverfügbar, performant und sicher** (Datenschutz/DSGVO, sichere Zahlung) sein.

Im Folgenden werden mit GQM Qualitätsziele über den **gesamten Entwicklungszyklus** (Anforderung, Entwurf, Implementierung, Test, Betrieb) abgeleitet; die Ziele orientieren sich an den geforderten Qualitätseigenschaften.

GQM-Modell

	Frage	Metrik
G1 – Gebrauchstauglichkeit aus Sicht der Kunden verbessern		
Q1	Wie reibungslos gelingt eine Buchung?	Task-Completion-Rate, Abbruchrate im Buchungs-Funnel, Time-on-Task
Q2	Wie verständlich ist der Ausflug-Konfigurator?	Klicks/Schritte bis zur fertigen Planung, SUS-Score
Q3	Funktioniert die Bedienung auf allen Geräten?	erfolgreiche Sitzungen je Browser/-Gerät
G2 – Zuverlässigkeit und Verfügbarkeit im Betrieb verbessern		
Q4	Wie verfügbar ist das Portal?	Uptime %, MTBF, MTTR
Q5	Wie korrekt arbeiten Filter-, Kollisions- und Verfügbarkeitsprüfung?	Fehlbuchungsrate, Quote korrekt erkannter Terminkollisionen
G3 – Performance und Reaktionsfähigkeit verbessern		
Q6	Wie schnell lädt das Portal/reagiert die UI?	Seitenladezeit (p95), Antwortzeit der Filterung, Time-to-Interactive
Q7	Wie verhält es sich bei Lastspitzen (Saison)?	Durchsatz, Antwortzeit unter Last
G4 – Sicherheit und Datenschutz gewährleisten		
Q8	Wie sicher sind Zahlung und Daten?	Anzahl offener Schwachstellen (CVSS), Anteil verschlüsselter Verbindungen, Pentest-Ergebnis
Q9	Wie schnell werden Lücken geschlossen?	MTTR für Sicherheitsfehler, Abdeckung der DSGVO-Anforderungen
G5 – Wartbarkeit des Codes aus Entwicklersicht verbessern		
Q10	Wie komplex/wartbar ist der Code?	Zyklomatische Komplexität, WMC, Code-Smell-Dichte
Q11	Wie gut ist der Code getestet?	Zeilen-/Branch-Coverage
G6 – Release-Prozess effizient und vorhersehbar gestalten (Prozess/Projekt)		
Q12	Wie viele Fehler erreichen die Produktion?	Escaped-Defects, Fehlerdichte je Release
Q13	Wie schnell/zuverlässig liefern wir?	Lead-Time, Deployment-Frequenz, Change-Failure-Rate

Interpretation der Metriken

Nutzen für die Qualitätsverbesserung. Die Metriken machen Qualität objektiv sichtbar, ermöglichen einen Soll-/Ist-Abgleich und erlauben es, Verbesserungen messbar zu belegen (z. B. Crash-Rate vor/nach einem Release).

Sichtbar gemachte Probleme/Risiken. Eine fehlerhafte Kollisions- oder Verfügbarkeitsprüfung (G2) führt zu Doppel- oder Fehlbuchungen; Schwachstellen bei Zahlung/Datenhaltung (G4) bedrohen Geld und persönliche Daten; Ausfälle oder lange Antwortzeiten (G2, G3) – gerade in der Hauptsaison – kosten Buchungen; hohe Komplexität bei niedriger Coverage (G5) deutet auf wartungsintensiven, fehleranfälligen Code.

Profitierende Stakeholder.

- *Kunde/Tourist:* einfache Planung, korrekte Buchungen, Schutz von Daten und Zahlung (G1–G4).
- *Ausflugsanbieter/Tourismus:* weniger Fehlbuchungen, höhere Verfügbarkeit in der Saison (G2).
- *Entwickler:* wartbarer Code, weniger Bugfixing (G5).
- *Betrieb (Ops):* überwachbare Verfügbarkeit/Performance (G2, G3).
- *Datenschutz/Compliance:* nachweisbare DSGVO-Konformität (G4).
- *Product Owner/Management:* planbare, fehlerarme Releases (G6).

Abgeleitete Maßnahmen (Beispiele).

- Fehlbuchungen / verpasste Kollisionen → gezielte Tests der Konfigurator-Logik, Korrektur der Filter-/Prüfregeln.
- Offene Schwachstellen → Pentests, Dependency-Scans, priorisiertes Patchen, Verschlüsselung erzwingen.
- Komplexität/Coverage über Schwellenwerten → Refactoring & zusätzliche Tests, Quality-Gate im CI.
- Schlechter SUS-Score / hohe Abbruchrate → Usability-Tests, Überarbeitung von Konfigurator und Buchungs-Flow.

Einsatz in der Praxis. Die Metriken werden weitgehend automatisiert erhoben (statische Analyse, Tests und Coverage im CI, Security-/Dependency-Scans, Verfügbarkeits- und Performance-Monitoring im Betrieb, Befragungen für Usability) und in einem **Dashboard** zusammengeführt. Über **Quality-Gates** und regelmäßige Auswertung in Retrospektiven wird die Qualität kontinuierlich gesteuert – so entsteht ein geschlossener Regelkreis aus Messen, Bewerten und Verbessern.

Aufgabe 2: Identitäts-Disharmonien (Kategorie 1)

Grundlagen

Lanza & Marinescu beschreiben guten objektorientierten Entwurf über drei “Harmonien”. **Identitäts-Disharmonien** betreffen die *Harmonie der Identität*, d.h. die Frage, ob ein **einzelnes Element (eine Klasse oder Methode) für sich genommen** gut definiert ist – unabhängig von seinen Beziehungen zu anderen Elementen. Eine Disharmonie liegt vor, wenn ein Element zu viel, zu komplex oder umgekehrt zu wenig (nur Daten) macht.

Typische Ursachen:

- schlechte Verteilung der Verantwortlichkeiten (Verletzung des Single- Responsibility-Prinzips),
- “prozedurales Denken” in OO-Sprachen (Logik in wenigen großen Methoden/Klassen, Daten getrennt von Verhalten),
- ungebremstes Wachstum ohne begleitendes Refactoring.

Auswirkungen auf Qualität, Wartbarkeit und Weiterentwicklung: geringe Kohäsion und hohe Komplexität erschweren Verständnis, Test und Änderung; Fehler häufen sich; Wiederverwendung und Erweiterung werden blockiert – die Software “verrottet” über die Zeit.

Konkrete Disharmonien

Zur Identitäts-Kategorie gehören die folgenden Disharmonien (Code Smells):

Brain Method

Eine Methode, die zu viel Funktionalität einer Klasse in sich zentralisiert. *Merkmale:* sehr lang, viele Verzweigungen, tief verschachtelt, greift auf viele Variablen zu. *Einfluss:* schwer verständlich und testbar, hohe Fehleranfälligkeit; klassischer “Long Method”-Smell.

God Class

Eine Klasse, die zu viel “Intelligenz” an sich zieht und die Arbeit anderer Klassen übernimmt.

Merkmale: sehr viele Methoden/Verantwortlichkeiten, greift stark auf fremde Daten zu, geringe Kohäsion. *Einfluss:* zentrale Engstelle, Änderungen wirken sich breit aus, schwer wartbar.

Brain Class

Eine komplexe Klasse, die eine oder mehrere Brain Methods enthält und damit übermäßig viel Logik bündelt – die “methodenseitige” Verwandte der God Class. *Einfluss:* ähnlich wie God Class, mit Fokus auf interne Komplexität.

Data Class

Eine Klasse, die nur Daten hält (Felder mit Gettern/Settern), aber kein nennenswertes Verhalten besitzt – ein “dummer Datenhalter”. *Merkmale:* viele öffentliche Attribute/Accessoren, sehr geringe funktionale Komplexität. *Einfluss:* das zugehörige Verhalten liegt verstreut in anderen Klassen (Tell-Don’t-Ask verletzt), was Kapselung und Kohäsion schwächt.

Zentrale Metriken (Lanza)

Abk.	Bedeutung
WMC	Weighted Method Count – Summe der Methoden-Komplexitäten
ATFD	Access To Foreign Data – Zugriffe auf fremde Daten
TCC	Tight Class Cohesion – Kohäsion der Klasse
CYCLO	Zyklomatische Komplexität einer Methode
LOC	Lines of Code
MAXNESTING	maximale Verschachtelungstiefe
NOAV	Number Of Accessed Variables
WOC	Weight Of Class – Anteil “echter” Funktionalität
NOPA / NOAM	Number Of Public Attributes / Accessor Methods

Erkennung und Gegenmaßnahmen

Erkennung (Detection Strategies)

Disharmonien werden über **Detection Strategies** erkannt – logische Verknüpfungen mehrerer Metriken mit Schwellenwerten:

- **God Class:** $ATFD > FEW$ und $WMC \geq VERY\ HIGH$ und $TCC < 1/3$.
- **Brain Method:** $LOC > 2 \cdot HighAvg$ und $CYCLO \geq HIGH$ und $MAXNESTING \geq SEVERAL$ und $NOAV > MANY$.
- **Brain Class:** sehr hohe Gesamtkomplexität (WMC), niedrige TCC und mindestens eine Brain Method.
- **Data Class:** niedriger WOC und viele öffentliche Attribute/Accessoren ($NOPA+NOAM > FEW$).

Werkzeuge: iPlasma/inFusion (Lanza), SonarQube, PMD, Checkstyle, NDepend, Understand sowie JDeodorant zur Smell-/Refactoring-Erkennung.

Gegenmaßnahmen und Refactoring

Vorbeugung: konsequente Anwendung des Single-Responsibility-Prinzips, "Tell, Don't Ask", Information Hiding, regelmäßige Code-Reviews sowie Metrik-Schwellenwerte als Quality-Gate im CI und kontinuierliches Refactoring.

Disharmonie	Geeignete Refactorings
Brain Method	Extract Method, Decompose Conditional, Replace Temp with Query, Introduce Parameter Object, Replace Method with Method Object
God Class / Brain Class	Extract Class, Extract Subclass, Move Method, Move Field (Verantwortlichkeiten aufteilen)
Data Class	Move Method (Verhalten zur Klasse holen), Encapsulate Field, Encapsulate Collection, Tell-Don't-Ask umsetzen